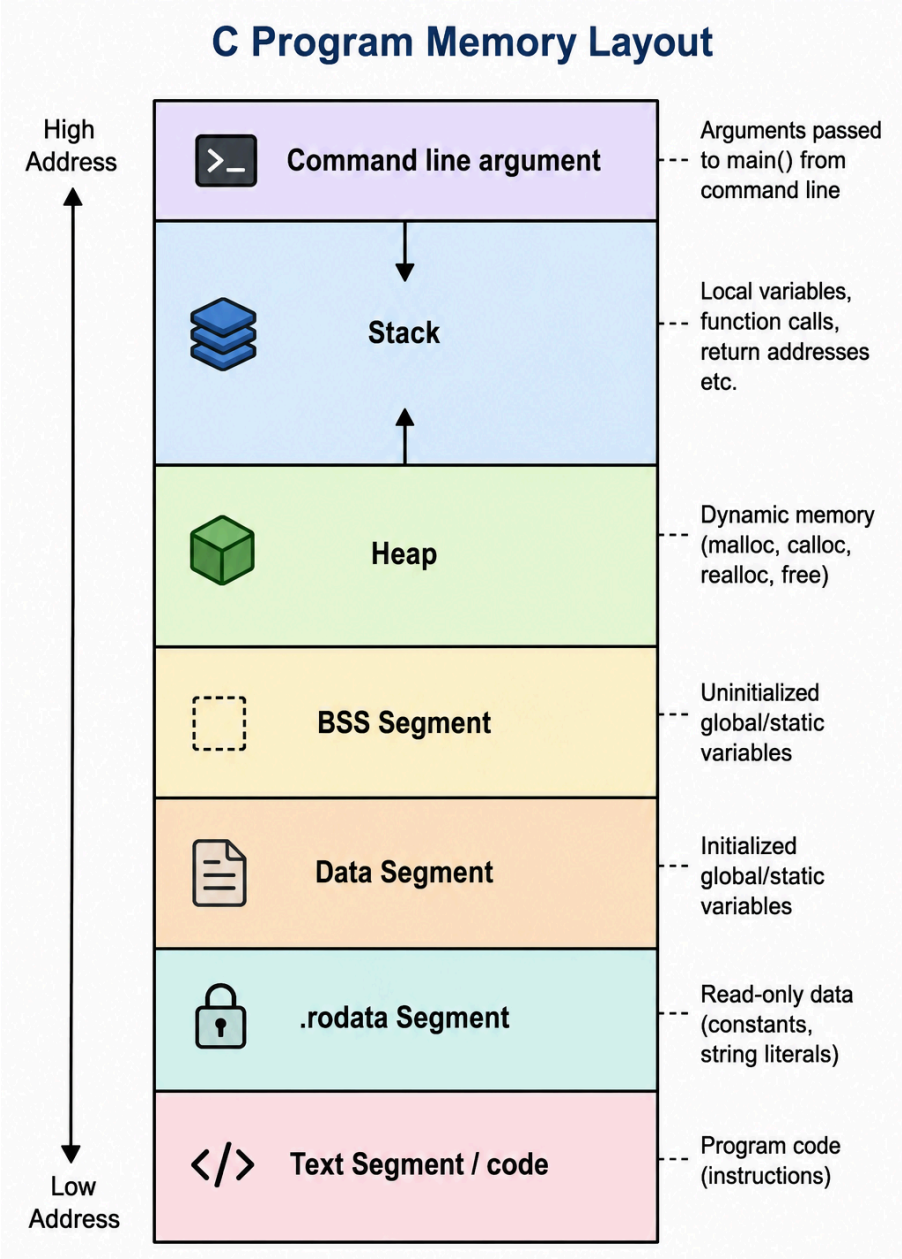
Memory Layout

Memory Layout in C.....1
1. Text Segment (Code Segment).....2
2. .rodata Segment (Read-Only Data).....3
3. Initialized Data Segment (.data).....7
4. Uninitialized Data Segment (BSS).....7
5. Heap Segment.....8
6. Stack Segment.....8
Practice Questions.....10

Memory Layout in C

Memory layout means **how a program and its data are arranged inside the computer's RAM (memory) while the program is running**.

When you run a C program, the operating system/compiler divides the program's memory into different sections. Each section has a specific purpose.



1. Text Segment (Code Segment)

Contains:

- Compiled machine instructions of the program

Properties:

- Read-only
- Prevents accidental modification

Example:

```
#include <stdio.h>

void fun() {
   printf("Hello");
}
```

The function fun() is stored in the **text segment**

2. .rodata Segment (Read-Only Data)

Contains:

- Constant values
- String literals

Properties:

- Read-only (modifying causes runtime error)

Example:

```
#include <stdio.h>

int main() {
   char *str = "Hello";
   printf("%s", str);
}
```

"Hello" is stored in **.rodata**, not in stack.
Trying str[0] = 'h'; → Undefined behavior (often crash).

What Gets Stored in .rodata

1. String Literals

Example: char *ptr = "Hello";

Here "Hello" is stored in a rodata because string literals are read-only.

Memory Mapping

Item	Section
pointer ptr	Stack
"Hello"	.rodata

Example

```
#include <stdio.h>

int main()
{
   char *ptr = "Hello";

   printf("%s", ptr);
}
```

2. Global const Variables

Example: const int a = 10;

Usually stored in **.rodata** because it is read-only.

Example

```c
#include <stdio.h>

const int a = 10;

int main()
{
   printf("%d", a);
}
```

3. Static const Variables

Example: static const int x = 100;

Stored in a **.rodata.**

4. Constant Arrays

Example: const char msg[] = "Embedded";

Stored mostly in .rodata because array contents are constant.

5. Compiler Generated Constant Data

Sometimes compiler stores:

- floating-point constants
- lookup tables
- format strings

Example: printf("Value = %d", x);

Format string "Value = %d" goes to .rodata.

What Does NOT Go Into .rodata

Local Variables int a = 5;

Stored on:

- stack

Global Initialized Non-const Variables

int g = 10;

Stored in a **.data** because variable is writable.

Uninitialized Globals

int x;

Stored in: .bss

Important Example

```c
#include <stdio.h>
const int g = 20;
int main()
{
   int a = 5;
   const int b = 10;
   char *ptr = "Hello";
   return 0;
}
```

Memory Layout

Variable	Section
g	.rodata
"Hello"	.rodata
ptr	Stack
a	Stack
b	Usually Stack

Why Local const Usually NOT in .rodata

Example: const int b = 10;

This is local variable. Compiler usually places it on stack because it belongs to function scope.

But compiler optimization MAY replace it with immediate constant.

Important Note

.rodata is:

- read-only
- protected by OS

Trying to modify it causes:

- segmentation fault
- access violation

Example

```
char *ptr = "Hello";

ptr[0] = 'A';   // ERROR
```

Reason:

- "Hello" stored in .rodata
- cannot modify read-only memory

3. Initialized Data Segment (.data)

Contains:

- Global and static variables initialized with values

Example:
int global = 10;
static int s = 20;

Both are stored in **data segment**

4. Uninitialized Data Segment (BSS)

Contains:

- Global and static variables **not initialized**

Default Value:

- Automatically initialized to **0**

Example:

```
int x;
static int y;
```

Stored in **BSS segment**

5. Heap Segment

Contains:

- Dynamically allocated memory

Functions Used:

- malloc(), calloc(), free()

Properties:

- Grows **upward**
- Managed manually by programmer

Example:

```c
#include <stdlib.h>

int main() {
   int *ptr = (int*) malloc(sizeof(int));
   *ptr = 100;
   free(ptr);
}
```

ptr is in stack, but allocated memory is in **heap**

6. Stack Segment

Contains:

- Local variables
- Function parameters
- Function call frames

Properties:

- Grows **downward**
- Automatically managed

Example:

```c
int main() {
   int a = 10;
}
```

Variable a is stored in **stack**

Combined Example

```c
#include <stdio.h>
#include <stdlib.h>

int global = 5;         // Data segment
int x;                  // BSS

int main() {
   int a = 10;         // Stack
   static int s = 20; // Data segment

   char *str = "Hi";  // str → Stack, "Hi" → .rodata

   int *ptr = malloc(sizeof(int)); // Heap
   *ptr = 50;

   printf("%d %d %d %s %d", global, s, a, str, *ptr);

   free(ptr);
   return 0;
}
```

Key Points

- **Text** → program instructions
- **.rodata** → constants, string literals
- **Data** → initialized global/static
- **BSS** → uninitialized global/static (default = 0)
- **Heap** → dynamic memory
- **Stack** → local variables, function calls

Practice Questions

1. Which memory section stores global initialized variables?

int x = 10;

A) Stack
B) Heap
C) Data Segment
D) BSS Segment

Answer: C) Data Segment

---

2. Which section stores uninitialized global variables?

int x;

A) Heap
B) Stack
C) BSS Segment
D) Code Segment

Answer: C) BSS Segment

---

3. Where is a local variable stored?

```
void fun()

{

  int a = 5;

}
```

A) Heap
B) Stack
C) Data Segment
D) Code Segment

Answer: B) Stack

---

4. Which memory grows dynamically during runtime using malloc()?

A) Stack
B) Heap
C) BSS
D) Text Segment

Answer: B) Heap

---

5. Which segment stores program instructions?

A) Stack
B) Heap
C) Text Segment
D) BSS

Answer: C) Text Segment

---

6. Predict the output

```
#include<stdio.h>


int x;


int main()

{

   printf("%d", x);

}
```

A) Garbage value
B) Runtime Error
C) 0
D) Compilation Error

Answer: C) 0

---

7. Which variable is NOT stored in stack memory?

A) Local variable
B) Function parameter

C) Static local variable
D) Automatic variable

Answer: C) Static local variable

---

8. What happens to stack memory after function return?

A) Deleted automatically
B) Remains forever
C) Moved to heap
D) Converted into BSS

Answer: A) Deleted automatically

---

9. Which memory allocation is faster?

A) Heap
B) Stack
C) Both same
D) Depends on compiler only

Answer: B) Stack

---

10. Which memory can cause fragmentation?

A) Stack
B) Heap
C) Text Segment
D) Register

Answer: B) Heap

---

11. Identify the segment

static int a = 5;

A) Stack
B) Heap

C) Data Segment
D) Code Segment

Answer: C) Data Segment

---

12. Identify the segment

static int a;

A) Stack
B) Heap
C) Data Segment
D) BSS Segment

Answer: D) BSS Segment

---

13. What is stored in stack besides local variables?

A) Function call information
B) Return address
C) Parameters
D) All of these

Answer: D) All of these

---

14. Which memory survives after function execution?

```
void fun()

{

  static int x = 0;

}
```

A) x destroyed
B) x survives
C) Compilation Error
D) Undefined behavior

Answer: B) x survives

---

15. What is the default value of static variables?

A) Garbage
B) NULL
C) 1
D) 0

Answer: D) 0

---

16. Which statement is TRUE?

A) Heap grows downward
B) Stack grows upward
C) Stack and heap grow towards each other
D) Heap and stack never change size

Answer: C) Stack and heap grow towards each other

---

17. Predict output

```
#include<stdio.h>
void fun()
{
   static int x;
   x++;
   printf("%d ", x);
}

int main()
{
   fun();
   fun();
   fun();
}
```

A) 1 1 1
B) 0 0 0

C) 1 2 3
D) Garbage values

Answer: C) 1 2 3

---

18. Which memory is managed manually by programmer?

A) Stack
B) Heap
C) Data Segment
D) Text Segment

Answer: B) Heap

---

19. What happens if heap memory is not freed?

A) Stack overflow
B) Memory leak
C) Syntax error
D) Deadlock

Answer: B) Memory leak

---

20. Which segment is usually read-only?

A) Stack
B) Heap
C) Text Segment
D) BSS

Answer: C) Text Segment

---

21. Predict output

#include<stdio.h>

```
int main()

{

   char *p = "Hello";

   p[0] = 'h';



   printf("%s", p);

}
```

A) hello
B) Hello
C) Compilation Error
D) Runtime Error / Undefined Behavior

Answer: D) Runtime Error / Undefined Behavior

---

22. Which memory allocation can cause stack overflow?

A) Excessive recursion
B) malloc()
C) calloc()
D) free()

Answer: A) Excessive recursion

---

23. Which variable has program-wide lifetime but local scope?

A) Global variable
B) Static local variable
C) Register variable
D) Heap variable

Answer: B) Static local variable

---

24. Which section contains constants and string literals?

A) Heap
B) Stack
C) Read-only data section
D) BSS

Answer: C) Read-only data section

---

25. Which memory is automatically allocated and deallocated?

A) Heap
B) Stack
C) Data Segment
D) BSS

Answer: B) Stack

---

26. Which variable is created only once?

A) Auto variable
B) Local variable
C) Static variable
D) Register variable

Answer: C) Static variable

---

27. What is wrong here?

```
int *fun()

{

  int x = 10;

  return &x;

}
```

A) Nothing wrong
B) x stored in heap

C) Returning address of stack variable
D) Syntax error

Answer: C) Returning address of stack variable

---

28. Which memory section size is fixed at compile time?

A) Stack only
B) Heap only
C) Data and BSS
D) Heap and Stack

Answer: C) Data and BSS

---

29. What happens when stack exceeds its limit?

A) Memory leak
B) Stack overflow
C) Heap corruption
D) Linker error

Answer: B) Stack overflow

---

30. Which is NOT part of process memory layout?

A) Stack
B) Heap
C) BIOS
D) Text Segment

Answer: C) BIOS